



HTML5

This content is protected and may not be shared, uploaded, or distributed.

What Are We Talking About Today

- Improved Markup and Semantics
 - Introduction of semantic tags: <header>, <footer>, <section>, <article>.
 - Enhanced document structure and readability.
- Graphics & Media
 - Video & Audio: <video> and <audio> for multimedia content.
 - Vector graphics & animations
 - Canvas: <canvas> for 2D graphics and support for WebGL.
 - WebGL, 3D graphics, WebXR
 - Support for modern video codecs like H.264, VP9, and AV1.
- Storage and APIs
 - Local and session storage for client-side data management.
 - File System API
 - Media capture & WebRTC
 - WebAuthN
 - WebWorkers
 - WebAssembly

Background

- HTML5 is the next major revision of HTML
- The Web Hypertext Application Technology Working Group (WHATWG) started work on the specification in June 2004
- HTML5 supersedes HTML 4.01, XHTML 1.0 and XHTML 1.1
- HTML5 vocabulary/APIs and canvas “Recommendations” available at <http://www.w3.org/TR/html5/> (redirected to WHATWG)
<http://www.w3.org/TR/2dcontext/>
- The latest versions of browsers have support for HTML5:
 - [http://en.wikipedia.org/wiki/Comparison_of_layout_engines_\(HTML5\)](http://en.wikipedia.org/wiki/Comparison_of_layout_engines_(HTML5))
- HTML 5.2, 2nd Edition “Recommendation” (Dec. 2017) available at <https://www.w3.org/TR/html52/>
- HTML Living Standard (6 April 2023) available at: <https://html.spec.whatwg.org/multipage/>
- W3C and WHATWG Agreement to collaborate on single version of HTML:
<https://www.w3.org/blog/news/archives/7753>

HTML5 - Removed Elements

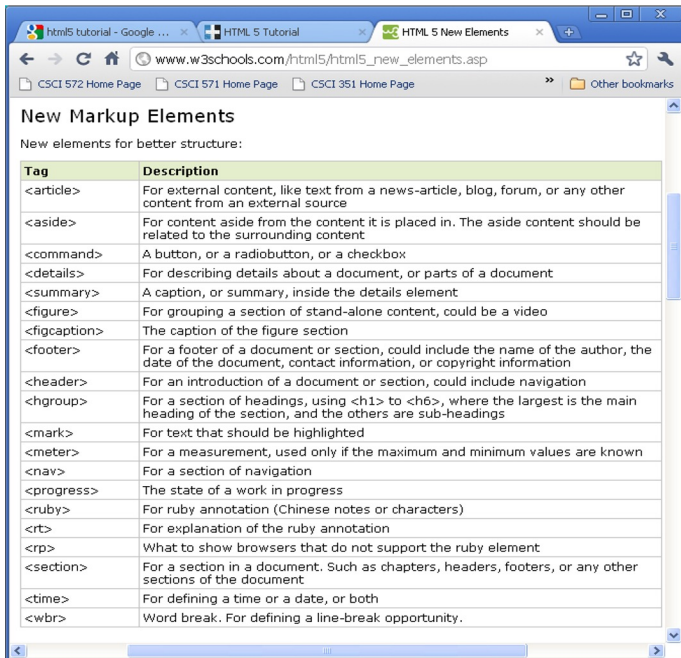
- The following elements are not in HTML 5 because their effect is purely presentational and therefore better handled by CSS:
 - *basefont, big, center, font, s, strike, tt, u*
- The following elements are not in HTML 5 because their usage affected usability and accessibility for the end user in a negative way:
 - *frame, frameset, noframes*
- *The following elements are not included because they have not been used often, created confusion or can be handled by other elements*
 - *acronym* is not included because it has created lots of confusion. Authors are to use *abbr* for abbreviations.
 - *applet* has been obsoleted in favor of *object*.
 - *isindex* usage can be replaced by usage of form controls.
 - *dir* has been obsoleted in favor of *ul*.

HTML5 - Removed Attributes - Handled by CSS

- align attribute on caption, iframe, img, input, object, legend, table, hr, div, h1, h2, h3, h4, h5, h6, p, col, colgroup, tbody, td, tfoot, th, thead and tr.
- alink, link, text and vlink attributes on body.
- background attribute on body.
- bgcolor attribute on table, tr, td, th and body.
- border attribute on table, img and object.
- cellpadding and cellspacing attributes on table.
- char and charoff attributes on col, colgroup, tbody, td, tfoot, th, thead and tr.
- clear attribute on br.
- compact attribute on dl, menu, ol and ul.
- frame attribute on table.
- frameborder attribute on iframe.
- height attribute on td and th.
- hspace and vspace attributes on img and object.
- marginheight and marginwidth attributes on iframe.
- noshade attribute on hr.
- nowrap attribute on td and th.
- Width attribute on hr, table, td, th, col, colgroup and pre
- scrolling attribute on iframe.
- size attribute on hr, input and select.
- type attribute on li, ol and ul.
- valign attribute on col, colgroup, tbody, td, tfoot, th, thead and tr.
- rules attribute on table.

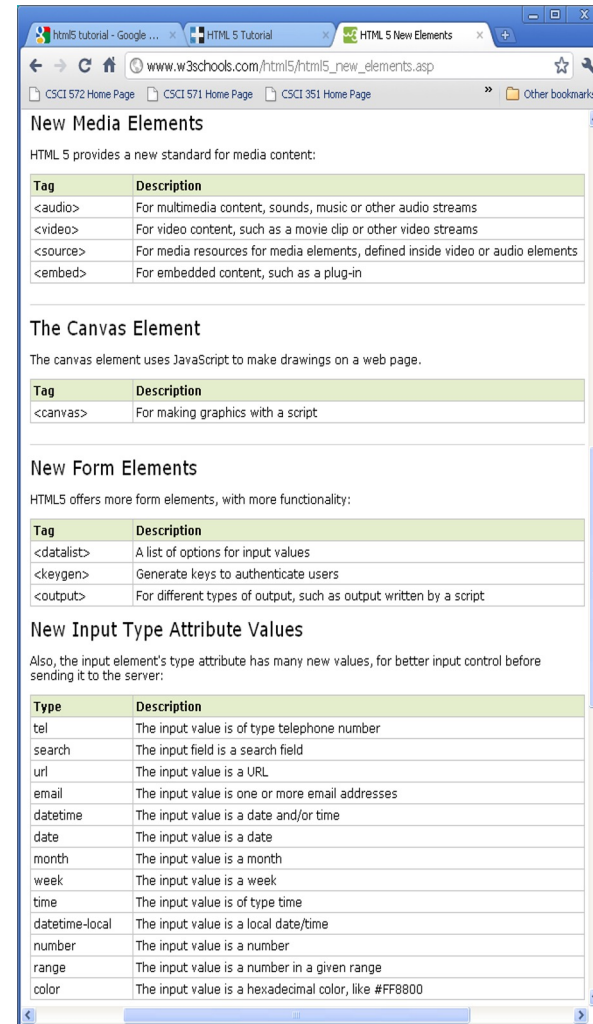
To simplify HTML many attributes have been eliminated as their effect is more properly produced using CSS

HTML5 - NEW Elements



New Markup Elements

<https://www.w3schools.com/html/default.asp>



New Media, Canvas, Form Elements

Semantic elements

- **Separation of Content and Presentation:** Semantic elements in HTML5 (e.g., `<header>`, `<article>`, `<footer>`, `<section>`, `<nav>`) promote a clear separation of content structure from styling, making it easier for developers to define a webpage's hierarchy and purpose. This fosters a more organized and maintainable codebase.
- **Improved Accessibility:** Semantic tags provide meaningful context to browsers, assistive technologies, and search engines. This improves accessibility for screen readers and enhances SEO by conveying the purpose of content elements more effectively.

SEO (Search Engine Optimization) is the process of making a website more visible in search results
- **Standardized Web Development:** The introduction of semantic elements encourages uniformity in web development, reducing the need for excessive `<div>` or `<span>` tags. This simplifies collaboration, debugging, and future development across teams.

Document Structure

```
<div id="header">
```

```
<div id="nav">
```

```
<div class="article">
```

```
<div class="section">
```

```
<div id="sidebar">
```

```
<div id="footer">
```

Old Way

```
<header>
```

```
<nav>
```

```
<article>
```

```
<section>
```

```
<aside>
```

```
<footer>
```

New Way

- HTML5 introduces a whole set of new elements that make it much easier to structure pages.
- By identifying the purpose of sections in the page using specific sectioning elements, assistive technology (e.g. for the blind) can help the user to more easily navigate the page.
- For example, they can easily skip over the navigation section or quickly jump from one article to the next without the need for authors to provide skip links.
- Authors also benefit because replacing many of the divs in the document with one of several distinct elements can help make the source code clearer, easier to author, and easier to scrape

See https://www.w3schools.com/html/html5_semantic_elements.asp

Defining Sections of a Web Page with <section> Element

- <section> - a grouping of content, e.g. chapters or tabbed pages or a page is divided into Introduction, News Items and Contact Information
- Example of an article about apples with two sections

```
<article> <hgroup>
```

```
<h1>Apples</h1>
```

```
<h2>Tasty, delicious fruit!</h2> </hgroup>
```

```
<p>The apple is the pomaceous fruit of the apple tree.</p>
```

```
<section> <h1>Red Delicious</h1>
```

```
  <p>These bright red apples are the most common found in many  
    supermarkets.</p> </section>
```

```
<section> <h1>Granny Smith</h1>
```

```
  <p>These juicy, green apples make a great filling for apple pies.</p>
```

```
</section>
```

```
</article> Don't use <section> just as hook for styling or scripting;
```

```
For that you should use <div>
```

```
Don't use <section> if <article>, <aside> or <nav> is more appropriate
```

```
Don't use <section> unless there is naturally a heading at the start of the  
section
```

Defining Sections of a Web Page with <nav> Element

- <nav> - a section of a page that links to other pages or to parts within the page; a section with navigation links
- In the following example, the page has several places where links are present, but only one of those places is considered a navigation section.

```
<body> <header>
<h1>Wake up sheeple!</h1>
<p><a href="news.html">News</a> -
    <a href="blog.html">Blog</a> -
    <a href="forums.html">Forums</a></p>
<p>Last Modified: <time>2009-04-01</time></p>
<nav> <h1>Navigation</h1>
    <ul>
        <li><a href="articles.html">Index of all articles</a></li>
        <li><a href="today.html">Things needed to wake up for today</a></li>
        <li><a href="successes.html">Sheep manage to wake</a></li>
    </ul>
</nav>
</header> <div>
<article> <header> <h1>My Day at the Beach</h1>
```

Defining Sections of a Web Page with <article> Element

- <article> - a component of a page that consists of a self-contained portion intended to be independently distributable or reusable;
 - This could be a forum post, a magazine or newspaper article, a blog entry, a user-submitted comment, an interactive widget or gadget, or any other independent item of content; Here is an example of a blog post showing an article followed by two comments:

```
<article> <header>
  <h1>The Very First Rule of Life</h1>
  <p><time pubdate datetime="2009-10-09T14:28-08:00"></time></p>
  </header>
  <p>If there's a microphone anywhere near you, assume it's hot and sending
  whatever you're saying to the world. Seriously.</p> <p>...</p>
<section> <h1>Comments</h1>
<article> <footer>
  <p>Posted by: George Washington</p>
  <p><time pubdate datetime="2009-10-10T19:10-08:00"></time></p>
  </footer>
  <p>Yeah! Especially when talking about your lobbyist friends!</p> </article>
<article> <footer>
  <p>Posted by: George Hammond</p> <p><time pubdate datetime="2009-10-10T19:15-
08:00"></time></p> </footer>
  <p>Hey, you have the same first name as me.</p>
</article> </section> </article>
```

HTML5 Media

HTML5 provides support for different media types

`<audio>` - for embedding and playing audio & music

`<video>` - for videos

`<svg>` - for vector images (in SVG format)

`<object>` - for external objects (pdf, etc.)

Video on the Web

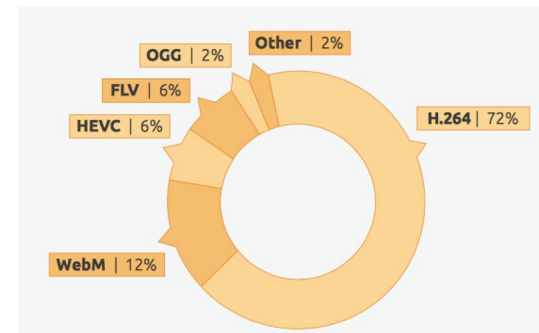
- HTML5 defines a standard way to embed video in a web page, using a <video> element.
- Video container files may include video, audio files and subtitles
- There are lots of competing container files, e.g.
 - Widely supported
 - **MPEG4** compressed video (mp4 or m4v)
 - **WebM** (VP8/VP9 video + Vorbis audio), from Google
 - Less common
 - QuickTime (.mov)
 - Flash Video (.flv) from Adobe (obsolete)
 - Ogg (.ogv) open source
 - Audio Video Interleave (.avi), invented by Microsoft

Video Codecs

- There are *lossy* and *lossless* video codecs. Lossless video is much too big to be useful on the web. A *lossy video codec* means that information is being irretrievably lost during encoding.
- Main codecs:
 - **H.264 (AVC)**
 - Most widely used codec for MP4.
 - Balances quality and compression efficiency.
 - **H.265 (HEVC)**
 - Successor to H.264 with better compression.
 - Ideal for 4K and HDR content.
 - **VP8/VP9**
 - Open-source codecs used with WEBM.
 - Efficient for web-based video delivery.
 - **AV1**
 - Open and royalty-free codec.
 - Designed for superior compression and next-gen video.
- Older codecs:
 - Theora (Open-source codec, part of the Ogg project.)
 - MPEG-2 (Primarily used for DVDs)

Video Codecs (recent development)

- The H.264 vs. WebM video “war” has ended.
- On March 2012 Firefox CTO “capitulated” and decided to support H.264. See: http://news.cnet.com/8301-30685_3-57397031-264/mozilla-execs-capitulate-in-h.264-web-video-war/
- HTML5 / H.264 now supported by YouTube. See: <http://www.youtube.com/html5>
- On March 7, 2013, Google admitted its VP8/WebM codec infringes MPEG H.264 patents, and agreed to pay to license H.264 patents, see <http://www.businesswire.com/news/home/20130307006192/en/Google-MPEG-LA-Announce-Agreement-Covering-VP8>
- As of January 2016, H.264 reigns supreme, Flash video on life support, see <https://www.encoding.com/blog/2016/01/27/h-264-and-hls-reign-in-online-video-finds-encoding-com-report/>
- Adobe system “retired” Flash at the end of 2020. See: <http://www.scmp.com/tech/enterprises/article/2104092/adobe-systems-retiring-flash-end-2020>
<https://www.adobe.com/products/flashplayer/end-of-life.html>

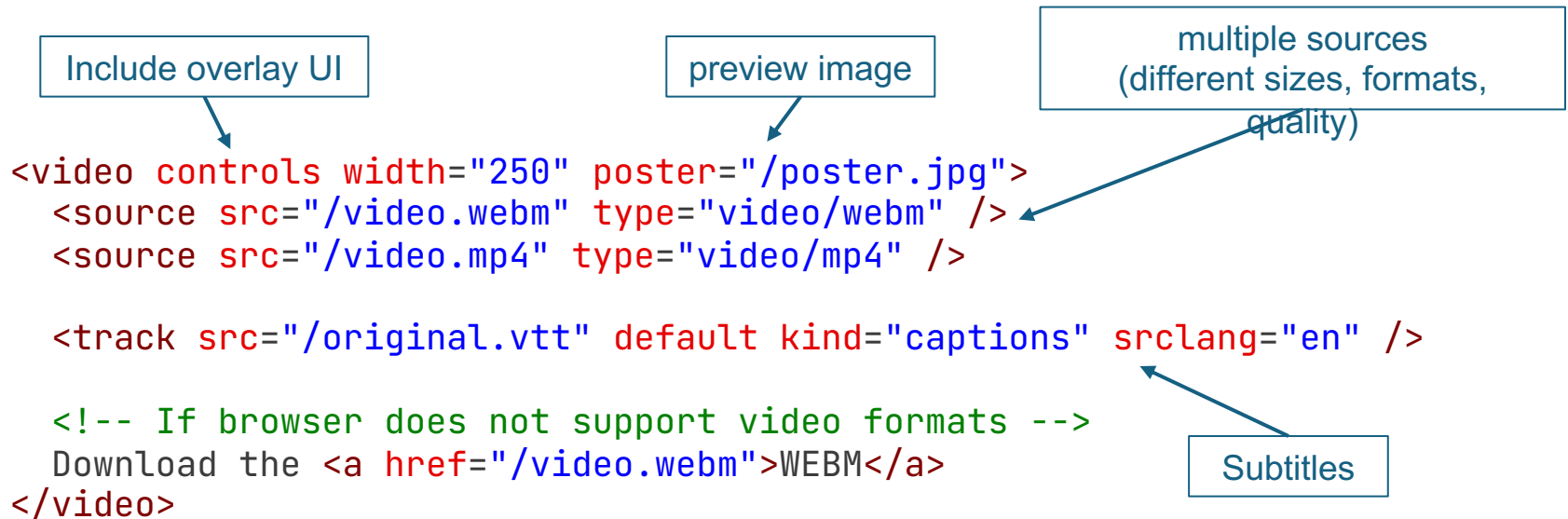


HTML Markup of Video

- To insert a video file in a web page, use the <video> element

```
<video src="pr6.webm" width="320" height="240"></video>
```

- The <video> element has methods like [play\(\)](#) and [pause\(\)](#) and a read/write property called [currentTime](#). There are also read/write [volume](#) and [muted](#) properties.
- you can tell the browser to display a built-in set of controls. To do this, just include the controls attribute in your <video> tag.



Using Attributes for Video

- The **preload** attribute tells the browser that you would like it to start downloading the video file as soon as the page loads. This makes sense if the entire point of the page is to view the video. On the other hand, if it's just supplementary material that only a few visitors will watch, then you can set `preload` to `none` to tell the browser to minimize network traffic.
- Here's an example of a video that will start downloading (but not playing) as soon as the page loads:

```
<video src="pr6.webm" width="320" height="240" preload></video>
```

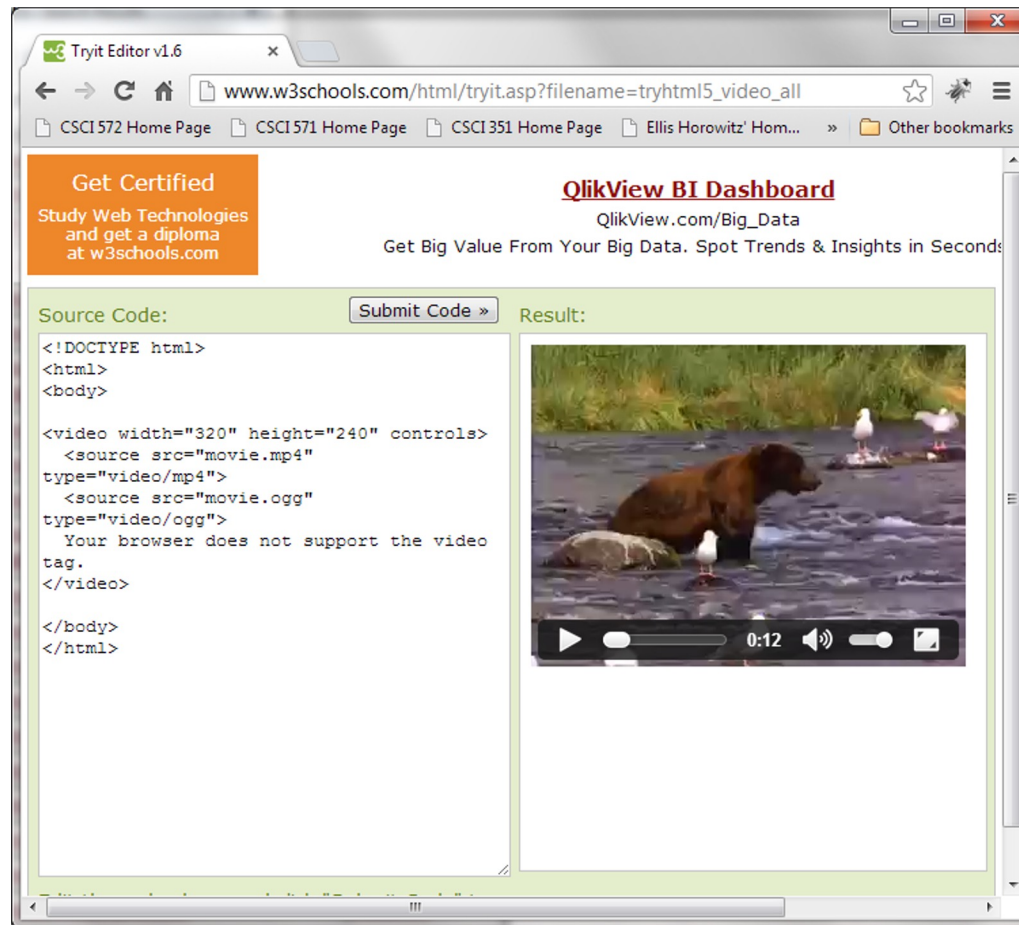
- And here's an example of a video that will *not* start downloading as soon as the page loads:

```
<video src="pr6.webm" width="320" height="240" preload="none"></video>
```

- The **autoplay** attribute tells the browser that you want to start downloading the video file as soon as the page loads, *and* you would like it to start playing the video automatically as soon as possible. Mostly blocked by browsers, unless muted.
- Here's an example of a video that will start downloading and playing as soon as possible after the page loads:

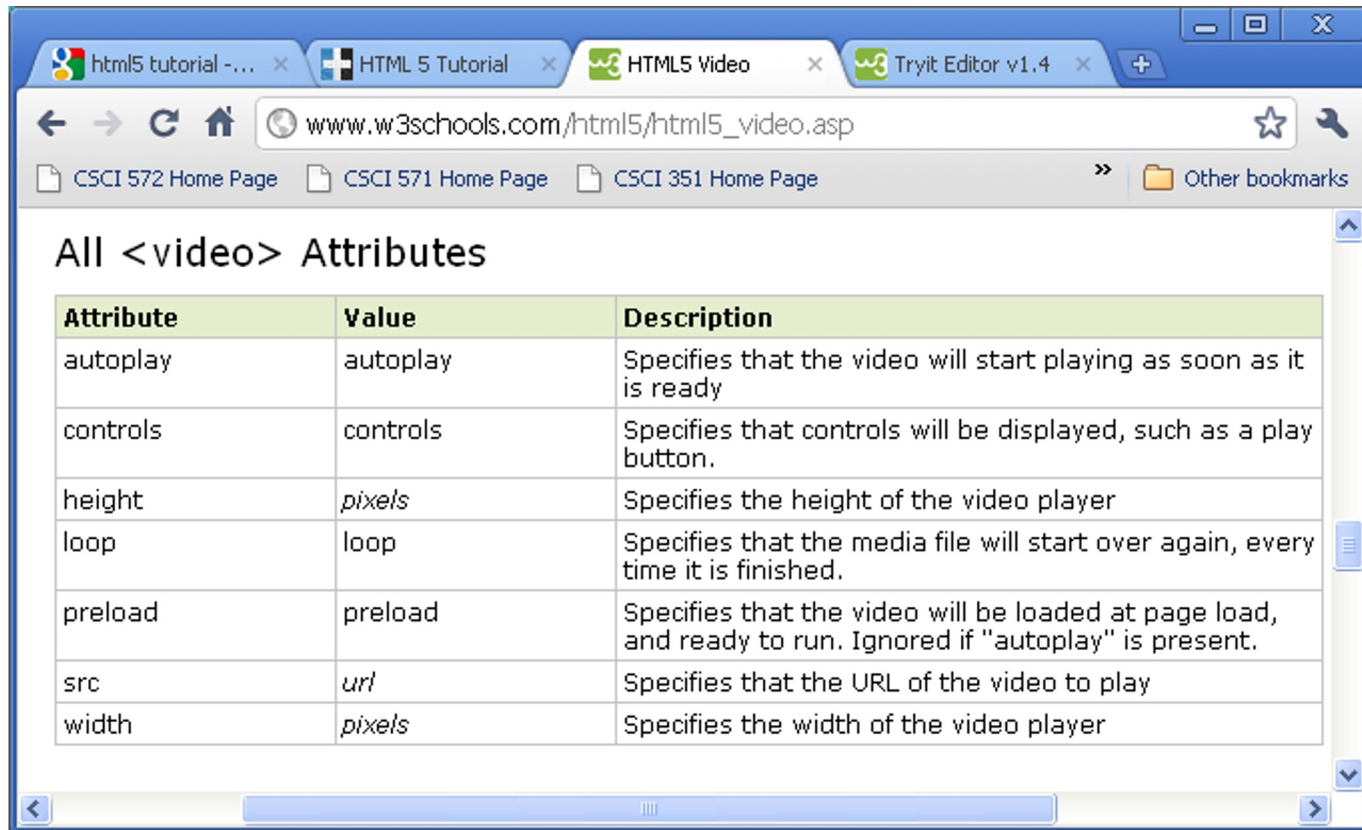
```
<video src="pr6.webm" width="320" height="240" autoplay></video>
```

HTML5 Video Example



http://www.w3schools.com/html/tryit.asp?filename=tryhtml5_video_all

Video Attributes



html5 tutorial -... x HTML 5 Tutorial x HTML5 Video x Tryit Editor v1.4 x

www.w3schools.com/html5/html5_video.asp

CSCI 572 Home Page CSCI 571 Home Page CSCI 351 Home Page Other bookmarks

All <video> Attributes

Attribute	Value	Description
autoplay	autoplay	Specifies that the video will start playing as soon as it is ready
controls	controls	Specifies that controls will be displayed, such as a play button.
height	<i>pixels</i>	Specifies the height of the video player
loop	loop	Specifies that the media file will start over again, every time it is finished.
preload	preload	Specifies that the video will be loaded at page load, and ready to run. Ignored if "autoplay" is present.
src	<i>url</i>	Specifies that the URL of the video to play
width	<i>pixels</i>	Specifies the width of the video player

Audio Codecs

A codec, short for “coder-decoder,” is technology used to compress and decompress video data, making it smaller for efficient streaming and storage

- And like lossless video, lossless audio is really too big to put on the web.
- The *audio codec* specifies how to decode the audio stream and turn it into digital waveforms that your speakers then turn into sound.
- On the web, there are really only three audio codecs you need to know about:
 - **AAC (Advanced Audio Codec)**
 - Modern format, supports up to 48 channels
 - Widely used in MP4 files and streaming platforms.
 - High-quality audio with efficient compression.
 - **MP3 (MPEG Audio Layer III)**
 - Old format, supports only up to 2 channels
 - One of the most popular and widely supported codecs.
 - **Opus**
 - Open-source and versatile codec optimized for internet use.
 - Excellent quality at low bitrates, suitable for VoIP and streaming.
 - **FLAC (Free Lossless Audio Codec)**
 - Lossless compression with no quality loss.
 - Ideal for audiophiles and archiving.
 - **Vorbis**
 - Open-source codec used with Ogg containers.
 - Good quality and compression, primarily for streaming.

modern .mp3 files are
actually in AAC
format

Audio Markup

control UI

looping support

```
<audio controls loop>
```

```
  <source src="myAudio.mp3" type="audio/mpeg" />
```

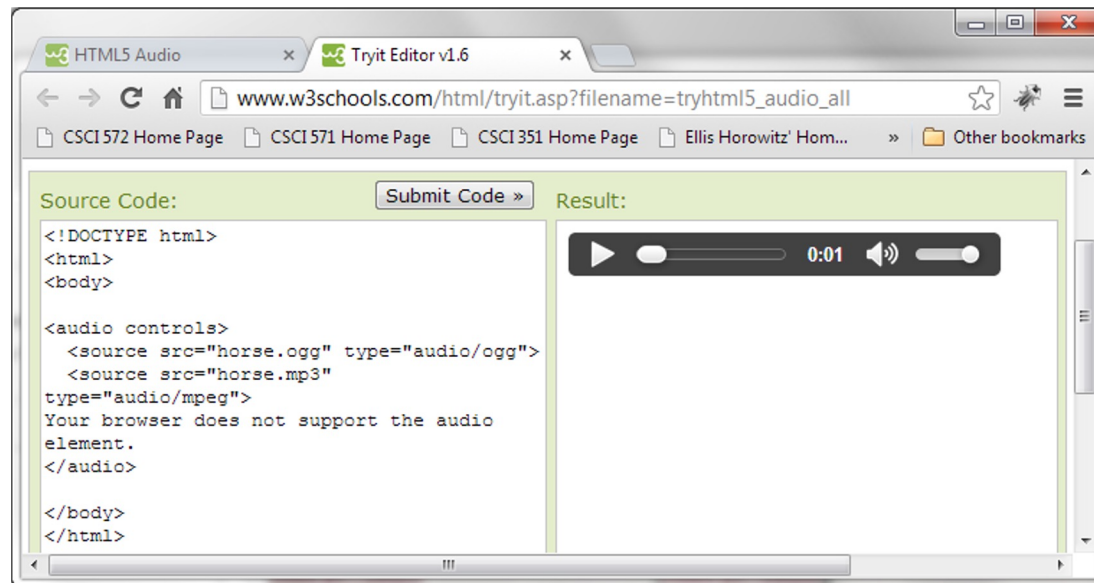
```
  <source src="myAudio.ogg" type="audio/ogg" />
```

multiple sources
(different formats, quality)

```
<!-- If browser does not support video formats -->
```

```
<p>Download <a href="myAudio.mp3" download="myAudio.mp3">MP3</a>.</p>
```

```
</audio>
```



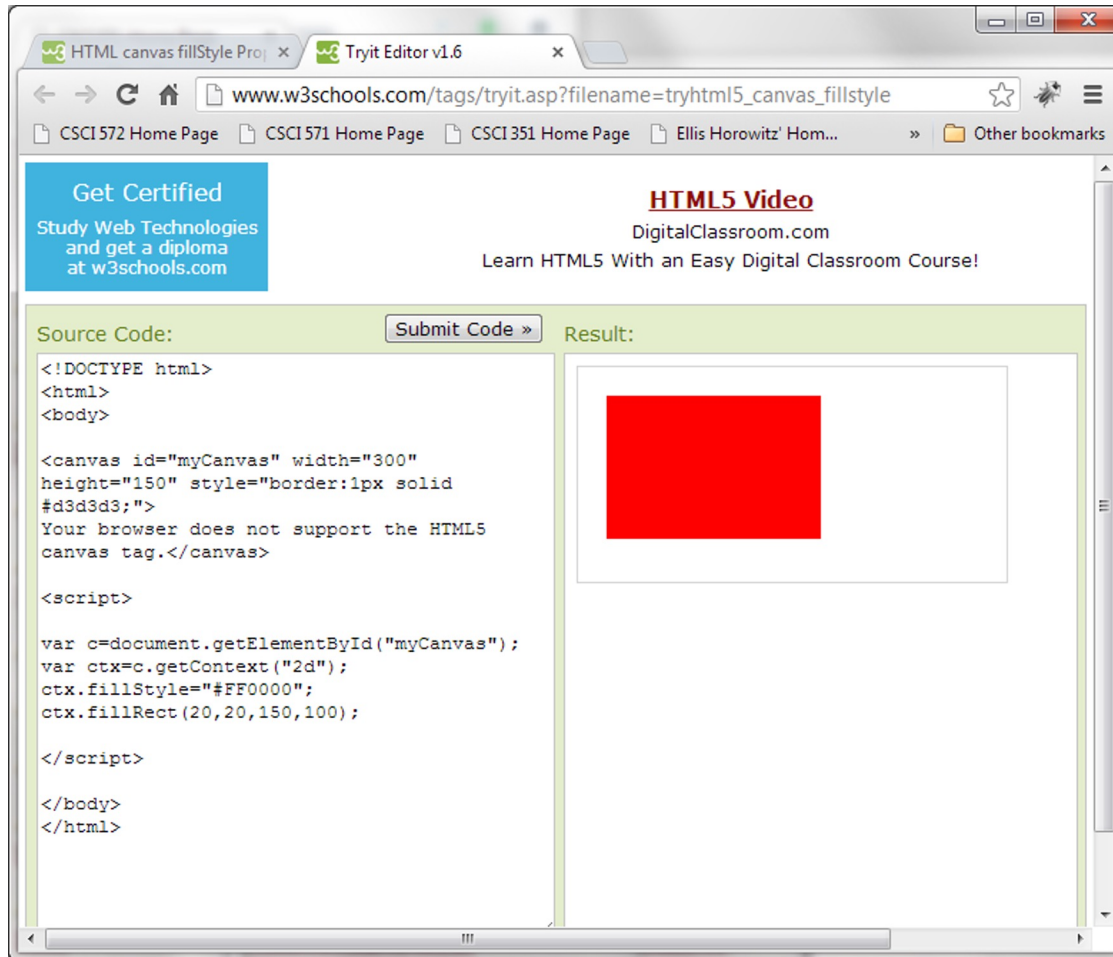
<canvas> Element – Low Level Drawing

- The <canvas> element is “a resolution-dependent bitmap canvas which can be used for rendering graphs, game graphics, or other visual images on the fly.”
- A *canvas* is a rectangle in your page where you can use JavaScript to draw anything you want.
- Canvas is a low-level API, that provides significant performance with direct GPU/Buffer API optimizations – **hardware accelerated**
- Each canvas will show up in the DOM, and each canvas maintains its own state. If you give each canvas an **id attribute**, you can access them just like any other element.
- E.g. if one adds an id attribute, e.g.

```
<canvas id="mycanvas" width="300" height="225"></canvas>
```
- Now you can easily find that <canvas> element in the DOM, e.g.

```
const mycanvas = document.getElementById("mycanvas");
```
- Can capture user events and translate them to objects using libraries

<canvas> element - Drawing a Red Rectangle



Every canvas has a drawing *context*.

Once you have found a <canvas> element in the DOM you call its `getContext()` method. You must pass the string "2d" to the `getContext()` method

JavaScript and DOM are used to create a red rectangle

https://www.w3schools.com/tags/tryit.asp?filename=tryhtml5_canvas_fillrect

<canvas> element - Coordinates

- The canvas is a two-dimensional grid. The coordinate (0, 0) is at the upper-left corner of the canvas. Along the X-axis, values increase towards the right edge of the canvas. Along the Y-axis, values increase towards the bottom edge of the canvas.
- To draw straight lines in pencil, you use the following two methods:
- **moveTo(x, y)** moves the pencil to the specified starting point.
- **lineTo(x, y)** draws a line to the specified ending point.

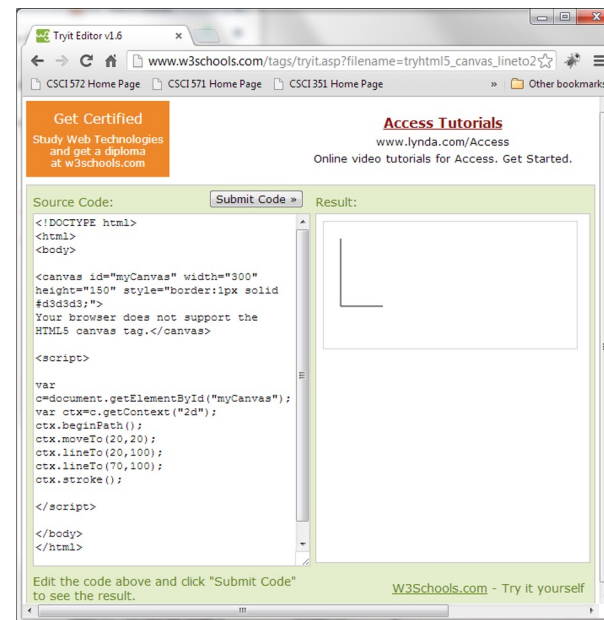
Move to (20,20)

Draw a line to (20,100)

Draw a line to (70,100)

stroke() makes the lines

Visible



https://www.w3schools.com/tags/tryit.asp?filename=tryhtml5_canvas_stroke

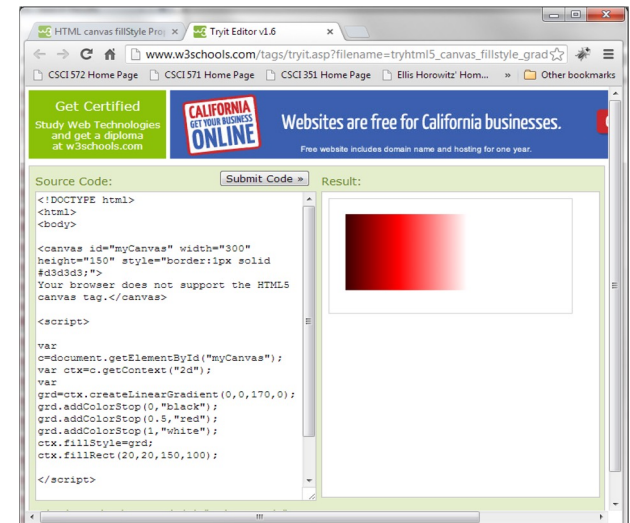
Drawing Context

- There's a whole group of properties and methods devoted to drawing rectangles:
- The fillStyle property can be a CSS color, a pattern, or a gradient. The default fillStyle is solid black, but you can set it to whatever you like.
- A gradient shows a difference in concentrations over an area, often using color intensity
- `createLinearGradient(x0,y0,x1,y1)` returns a canvas gradient object starting at (x0,y0) and ending at (x1,y1)
- `addColorStop(offset, color)` adds a color stop with the given color to the gradient at the offset; 0 offset is at one endpoint and 1 offset is at the other endpoint
- `fillRect(x, y, width, height)` draws a rectangle filled with the current fill style.
- `clearRect(x, y, width, height)` clears the pixels in the specified rectangle.

See <http://www.colorzilla.com/gradient-editor/>
for the Ultimate CSS Gradient Generator
or CSS Gradients in HTML5
<http://www.fix-css.com/2011/07/css-gradients-in-html5/>

Code to the right first defines a linear gradient region;
AddColorStop is then used to set black at the left endpoint
and white at the right endpoint; fillStyle is a context
property that when set to grd causes the color to fill the
region as shown

See http://www.w3schools.com/html/tryit.asp?filename=tryhtml5_canvas_tut_grad

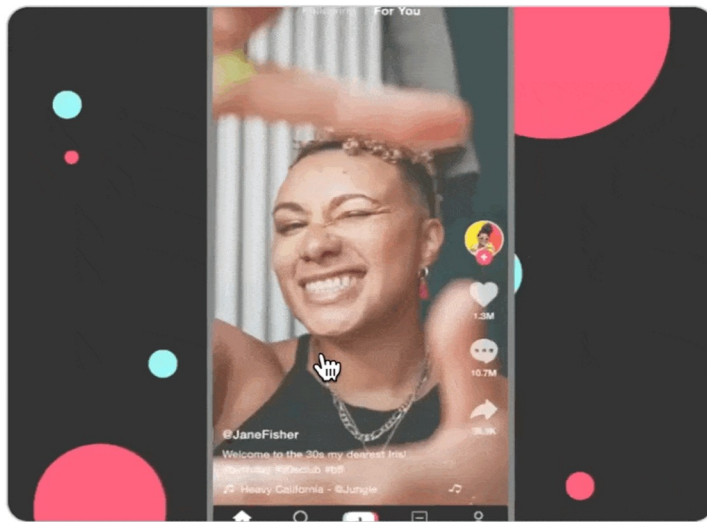


Power of canvas

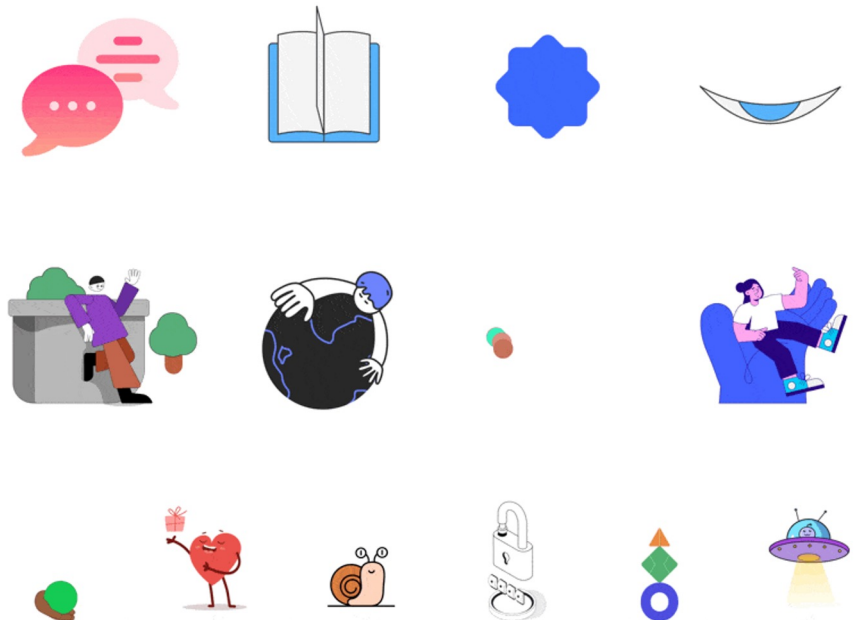
- Combined with WebGL could be used to output buffers from GPU
- Provides a way to render 3D, video games, charts, etc...
- Allows various online applications:
 - Gaming
 - Photo / video editors
 - Photoshop online
 - Visualizations and Charts
 - Apache Echarts <https://echarts.apache.org/examples/en/index.html#chart-type-globe>
 - D3.js <https://d3js.org/>
 - High performance
 - Google Spreadsheets
 - Maps
 - Apple Maps
 - Google Maps

SVG animations - Lottie

- Popular format for SVG-based animation & motion graphics
- Used in web, mobile applications (Web, iOS, Android, React Native, Windows...)
- Originally developed by AirBNB (<https://airbnb.io/lottie/#/>)
- Can call animations on user events – clicks, hovers, etc.
- Used by lots of applications
- <https://lottiefiles.com/> - large collection of public animations



 TikTok



Vector graphics - SVG

- The <svg> element is “a container for SVG graphics.”
- SVG has methods for drawing paths, boxes, circles, text and graphviz images.
- SVG supports animations, filters, effects, etc...
- SVG graphics is supported by all major browsers.
- SVG is a language for describing 2D graphics in XML.
- Since SVG is XML based, every element is available in the SVG DOM.
- https://www.w3schools.com/html/tryit.asp?filename=tryhtml_svg_circle

Canvas	SVG
• Resolution dependent • No support for event handlers • Poor text rendering capabilities • You can save the resulting image as .png or .jpg • Well suited for graphic-intensive games	• Resolution independent • Support for event handlers • Best suited for applications with large rendering areas (Google Maps) • Slow rendering if complex (anything that uses the DOM a lot will be slow) • Not suited for game applications

Comparison of Canvas and SVG

WebGL

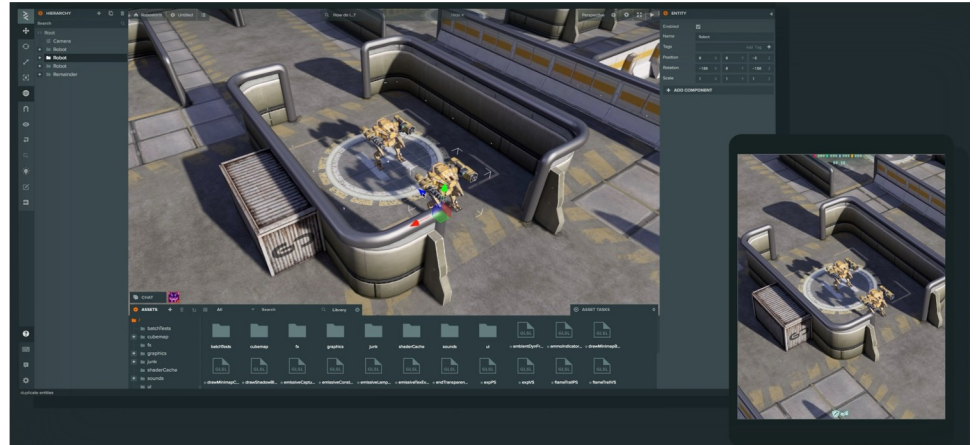
- **WebGL (Web Graphics Library)** is a JavaScript API for rendering high-performance interactive 3D and 2D graphics within any compatible web browser without the use of plug-ins.
- WebGL introduces an API that closely conforms to OpenGL ES 3.0 that can be used in HTML <canvas> elements.
- This conformance makes it possible for the API to take advantage of hardware graphics acceleration provided by the user's device.
- Widely supported by most modern browsers on both desktop and mobile platforms. WebGL is built into the browser, and no additional plugins are required.
- **OpenGL shaders support**
- Some examples what OpenGL shaders can do: (you can play with them)
 - <https://www.shadertoy.com/view/mtYGWY> (has a youtube tutorial)
 - <https://www.shadertoy.com/view/NsIGRN> (drag-interactive)
 - <https://www.shadertoy.com/view/X3sXRr>
 - <https://www.shadertoy.com/view/XsBXWt>
- More examples on <https://www.shadertoy.com>

3D Graphics

- **Canvas** also can help to provide 3D experiences for your website or AR/XR experiences with 3D headsets (e.g., Apple Vision Pro)
- Some popular JavaScript libraries:
 - <https://threejs.org/>
 - <https://tresjs.org/>
 - <https://www.babylonjs.com/>
- Some notable examples:
 - <https://tresjs.org/> (scroll down)
 - https://threejs.org/examples/#webgl_animation_keyframes
 - https://threejs.org/examples/#webgl_animation_skinning_ik

Gaming

- You can build true 3D-games for web browser
- Notable full-scale online 3D game engine + IDE:
<https://playcanvas.com/>



<https://robostorm.io/> (WASD + mouse)

- More examples: <https://playcanvas.com/explore#!>

WebVR, WebXR

- **WebVR** has been replaced by **WebXR**
- **WebXR** is a group of standards which are used together
 - to support rendering 3D scenes to hardware designed for presenting virtual worlds (virtual reality, or VR)
 - for adding graphical imagery to the real world, (augmented reality, or AR).
- The WebXR Device API implements the core of the WebXR feature set, managing the selection of output devices, render the 3D scene to the chosen device at the appropriate frame rate, and manage motion vectors created using input controllers.
- See <https://immersiveweb.dev/>
- https://developer.mozilla.org/en-US/docs/Web/API/WebXR_Device_API
- Explainer: <https://github.com/immersive-web/webxr/blob/master/explainer.md>
- Spec: <https://www.w3.org/TR/webxr/>

WebXR Support

- Desktop Browsers

- Google Chrome: supported starting from Chrome 79.
- Mozilla Firefox: support started with Firefox Reality and is integrated into Firefox Desktop since version 79.
- Microsoft Edge: Chromium-based Edge supports WebXR from version 79.

- Mobile Browsers

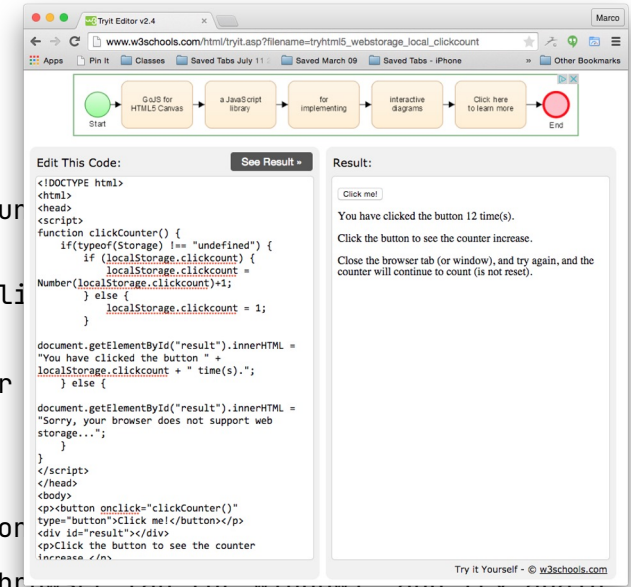
- Chrome for Android; WebXR is supported starting from version 79. Works with Android devices supporting ARCore.
- Firefox Reality: A specialized version of Firefox for AR and VR experiences.
- Samsung Internet Browser: Supports WebXR starting from version 11.2 for Android devices.
- Safari on iOS: Limited WebXR support. Apple focuses more on ARKit and may require custom apps for full XR functionality.

localStorage

- localStorage is a client-side key-value database,
 - data is stored in the user's browser and remains there **even across sessions** (open/close browser)
 - data is only available when on that machine and in that browser.
 - localStorage is per browser not per computer.
 - localStorage only supports storing of strings

- Below is an example of local storage that counts the number of clicks on a button

```
<!DOCTYPE html>
<html>
<head>
<script>
function clickCounter() {
    if(typeof(Storage) !== "undefined") {
        if (localStorage.clickcount) {
            localStorage.clickcount = Number(localStorage.clickcount) + 1;
        } else {
            localStorage.clickcount = 1;
        }
        document.getElementById("result").innerHTML = "You have clicked the button " +
            localStorage.clickcount + " time(s).";
    } else {
        document.getElementById("result").innerHTML = "Sorry, your
browser does not support web storage...";
    }
}
</script>
</head>
<body>
<p><button onclick="clickCounter()" type="button">Click me!</button>
<div id="result"></div>
<p>Click the button to see the counter increase.</p><p>Close the browser tab (or window), and try again,
and the counter will continue to count (is not reset).</p></body></html>
```



See: http://www.w3schools.com/html/tryit.asp?filename=tryhtml5_webstorage_local_clickcount

sessionStorage

- Session storage is designed for scenarios where the user is carrying out a single transaction but could be carrying out multiple transactions in different windows at the same time.
- To address this, this specification introduces the sessionStorage IDL attribute (An IDL attribute determines the behavior of script data). Sites can add data to the session storage, and *it will be accessible to any page from the same site opened in that window. The sessionStorage object is equal to the localStorage object, except that it stores the data for only one session. The data is deleted when the user closes the specific browser tab.*
- For example, a page could have a checkbox that the user ticks to indicate that he wants insurance:

```
<label>
```

```
<input type="checkbox" onchange="sessionStorage.insurance = checked ? 'true' : ''>
```

```
I want insurance on this trip.</label>
```

- A later page could then check, from script, whether the user had checked the checkbox or not:

```
if (sessionStorage.insurance) { ... }
```

- If the user had multiple windows opened on the site, each one would have its own individual copy of the session storage object.

Status on WebSQL

- **WebSQL** was only ever implemented in Blink-based browsers like Google Chrome and the new Microsoft Edge, and WebKit-based browsers like Safari.
- Web SQL was deprecated and removed for third-party contexts in Chromium 97. The API will be entirely removed from Chrome 124 (current version is 123).
- As of February 2024, WebSQL is being phased out in favor of **WebStorage** (localStorage and sessionStorage) and **IndexedDB**, but still available in some contexts under restrictive conditions.
- The Indexed Database API (commonly referred to as **IndexedDB**) is a JavaScript application programming interface (API) provided by web browsers for managing a **NoSQL database** of objects. It is a standard maintained by the World Wide Web Consortium (W3C).
- See Indexed Database API 3.0
<https://www.w3.org/TR/IndexedDB-3/>

File System API

- Provides API to work with OS files & directories - **read**, **write**, and **save** changes directly to files and directories on the user's device
- Users must explicitly **grant permissions** for each file or directory operation, ensuring privacy and security.
- Permissions are session-specific.
- Developers can create applications (e.g., text editors or image editors) that feel more like native desktop applications by reducing the need for repetitive upload/download steps.
- Integration with drag-and-drop and file picker APIs enhances usability.
- Support for this API is still evolving, **adoption varies among browsers**
- Full support: Chrome 86+, Edge 86+
- Partial support: Safari (WebKit), Safari implements a subset of the File System Access API features via the File and Directory Entries API.
- Not fully supported: Mozilla Firefox does not natively support the File System Access API but provides some similar functionality through file input controls or experimental APIs.
- Example: <https://mburakerman.github.io/file-system-access-api-demo/>
- MDN: https://developer.mozilla.org/en-US/docs/Web/API/File_System_API

Web Serial API

- Provides API to work with **USB devices**
- You can update devices' firmware, send files, receive data, etc.
- You can filter devices by their USB vendor ids
- **Only supported by Chrome and Edge** (Chromium-based)

```
// Connect to `e.target` or add it to a list of available ports.
navigator.serial.addEventListener("connect", (e) => {...});

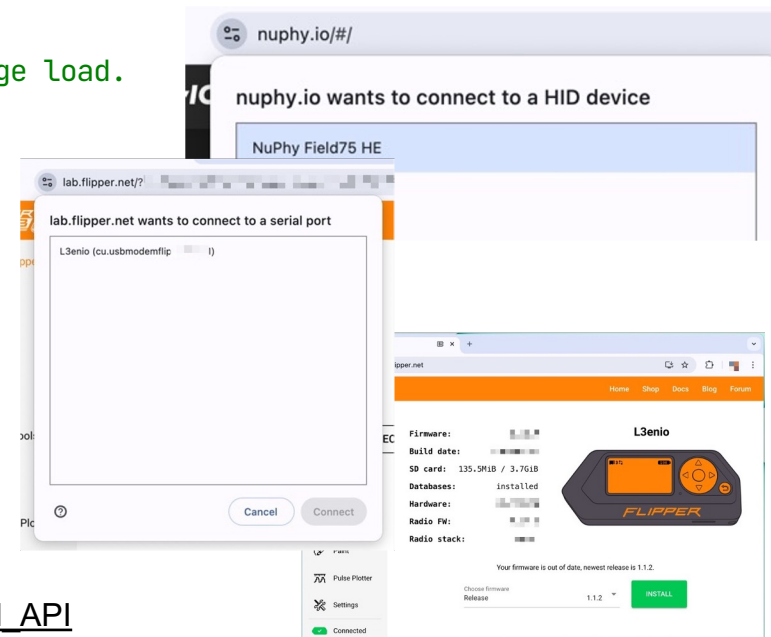
// Remove `e.target` from the list of available ports.
navigator.serial.removeEventListener("disconnect", (e) => {...});

// Initialize the list of available ports with `ports` on page load.
navigator.serial.getPorts().then((ports) => {...});

button.addEventListener("click", () => {
  navigator.serial
    .requestPort({ filters: [{ usbVendorId: 0xabcd }] })
    .then((port) => {...});
});
```

- Examples:

- <https://tigoe.github.io/html-for-conndev/webSerial/>
- <https://lab.flipper.net/>
- <https://nuphy.io/>
- MDN: https://developer.mozilla.org/en-US/docs/Web/API/Web_Serial_API

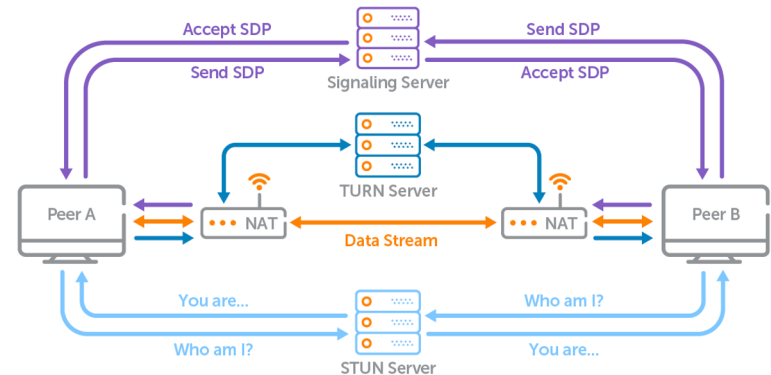


Media Capture and Streams API

- Provides methods to **capture** and **manipulate audio and video** streams
- **Widely supported across modern browsers.**
- Captures streams from user devices, such as **cameras** and **microphones**, and **screen-sharing tools** for audio and video capture, enabling real-time media applications like **video conferencing or recording**.
- MediaStream objects represent a collection of media tracks (audio or video), each consisting of one or more channels (e.g., stereo audio).
- These tracks can be manipulated independently for advanced use cases.
- Device Integration via `getUserMedia()`: Streams can be created locally from a user's device through the `getUserMedia()` method, while remote or network-based streams are integrated using WebRTC or other APIs like Web Audio.
- Can serve as input to various components:
 - `<video>` and `<audio>`
 - WebRTC connections
 - JS – for media recording or audio processing nodes in the Web Audio API.
- Examples: Google Meet, Zoom, etc.
- MDN: https://developer.mozilla.org/en-US/docs/Web/API/Media_Capture_and_Streams_API

WebRTC

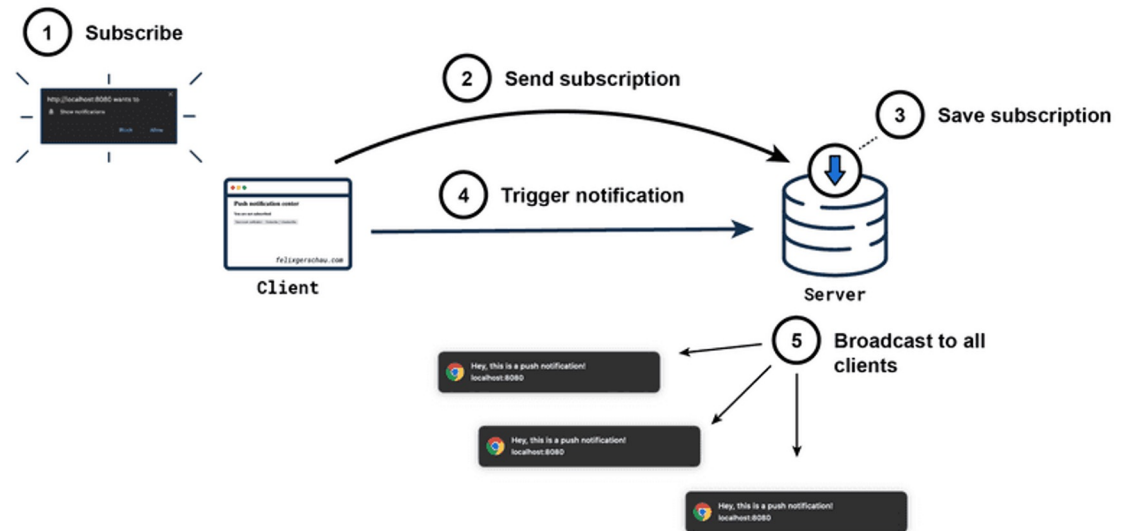
- **Real-Time Peer-to-Peer Communication**
- **Can penetrate NAT** with special technique (see TURN servers, <https://getstream.io/glossary/turn-server/>)
- WebRTC enables direct audio, video, and data communication between browsers or devices without requiring intermediaries, making it ideal for video calls, live streaming, and file sharing.
- Supports
 - high-quality **media streaming** (audio and video)
 - bidirectional **data transfer** WebRTC uses encryption (DTLS and SRTP) for secure communication
- **Supported by all major modern browsers**, enabling peer-to-peer communication for audio, video, and data sharing directly in the browser without additional plugins.



- Examples: Google Meet, Zoom, etc...
- See: <https://webrtc.github.io/samples/>
- MDN: https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API

Push API / Web Notifications

- Displays OS-native notifications
- **Push API** gives web applications the ability to receive messages pushed to them from a server (whether or not the web app is in the foreground, or even currently loaded).
- Delivers asynchronous notifications and updates to users that opt in



- Supported by Push service: (eg. OneSignal <https://onesignal.com/web-push>)
- **Widely supported across most modern browsers**
- MDN: <https://developer.mozilla.org/en-US/docs/Web/API/Notification>
- Web Dev: <https://web.dev/explore/notifications>

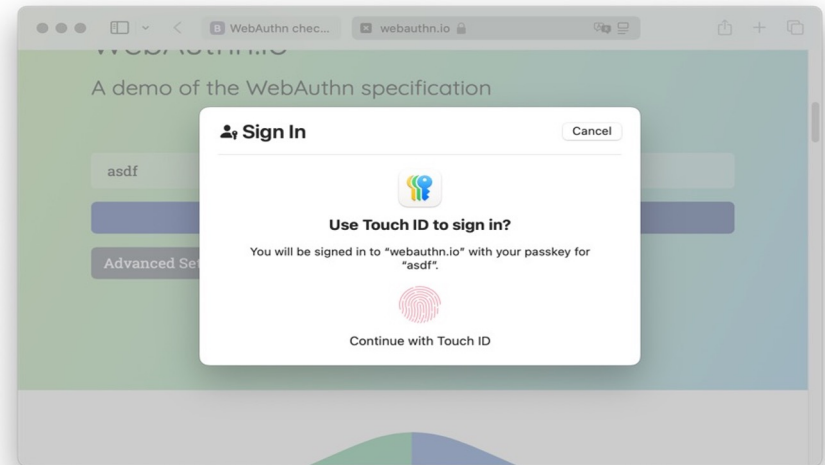
WebAuthN

- Enables passwordless authentication
- **Supported by most modern browsers.** It provides a secure, passwordless authentication mechanism using public key cryptography and is part of the FIDO2 standard.
- Uses **usb-tokens**, device authn features (**Fingerprint, TouchId, FaceId, Iris, etc.**)
- WebAuthn uses asymmetric (public-key) cryptography
- Protection against phishing: An attacker who creates a fake login website can't login as the user because the signature changes with the origin of the website.
- Reduced impact of data breaches
- Invulnerable to password attacks

✓ Baseline Widely available



This feature is well established and works across many devices and browser versions. It's been available across browsers since September 2021.



Demo: <https://webauthn.io/>

MDN: https://developer.mozilla.org/en-US/docs/Web/API/Web_Authentication_API

Web Workers

- **Widely supported by all major modern browsers.** They allow you to **run JavaScript in a separate thread**, which improves performance by preventing heavy computations from blocking the main UI thread.
- Make possible to move heavy processing off the main thread (run a script operation in a background thread separate from the main execution)
- Workers cannot access DOM directly
- **Dedicated workers** are workers that are utilized by **a single script**.
- **Shared workers** are workers that can be utilized by **multiple scripts** running in different windows, iFrames, etc., if they are in the same domain as the worker.
- **Service Workers** essentially act as **proxy servers** that sit between web applications, the browser, and the network (when available).

They are intended, among other things, to enable the creation of effective **offline experiences**, **intercept** network requests and take appropriate action based on whether the network is available, and update assets residing on the server. They will also allow access to push notifications and background sync APIs.

- MDN: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Web Assembly (WASM)



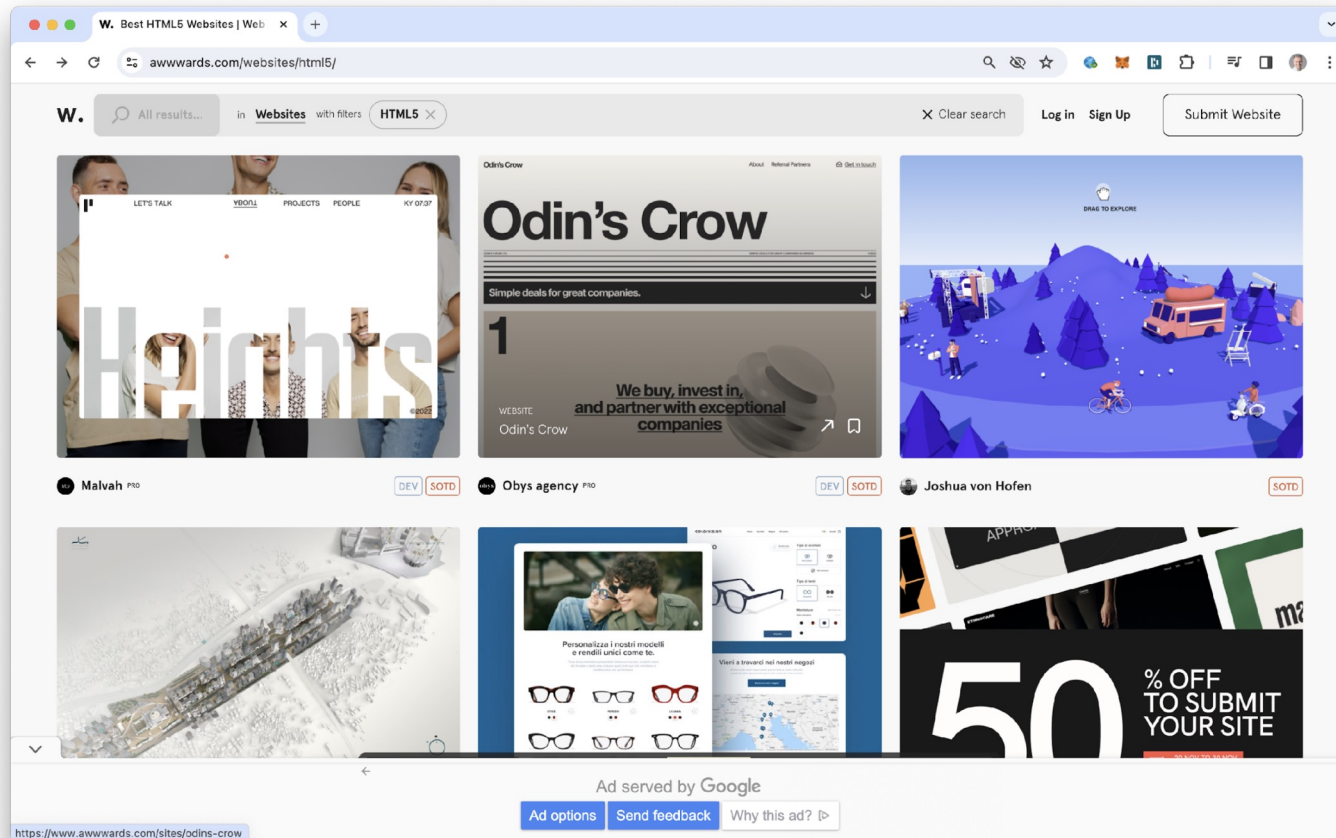
WEBASSEMBLY

- **Supported by all major modern browsers**, making it a widely accessible technology for running high-performance, low-level code on the web.
- Binary instruction format designed to run on **any platform**.
- Effectively allows to **run C/C++/Rust/... code in browser** to offload heavy processing to more performant languages
- WASM is designed for near-native execution speed, leveraging its compact binary format and low-level bytecode structure. It enables faster load times and efficient execution, making it ideal for compute-intensive applications like gaming, 3D modeling, and scientific simulations.
- Integrates seamlessly with JavaScript, allowing developers to call WebAssembly modules from JavaScript and vice versa. It operates in a secure sandboxed environment, safeguarding the host system from potential vulnerabilities
- Use cases:
 - Notion.io uses SQLite compiled to WASM in the browser to work with user data
 - Online code playground use WASM to run your non-js code in the browser
 - Google Meet uses WASM to blur the background and apply filters to video streams
 - Discord uses WASM for online in-browser noise cancellation
 - You can run ML models in WASM
- Website: <https://webassembly.org/>
- MDN: <https://developer.mozilla.org/en-US/docs/WebAssembly>

Best HTML5 Websites

- Some creative html5 websites for inspiration

<https://www.awwwards.com/websites/html5/>



More HTML5 fun....

- <http://www.chromeexperiments.com/>
- <http://www.creativebloq.com/web-design/examples-of-html-1233547>
- <https://www.noupe.com/essentials/freebies-tools-templates/40-beautiful-free-html5-css3-templates.html>
- <http://html5gallery.com>
- <https://www.juicebox.net/demos/>
- <http://wowslider.com/html5-gallery-puzzle-collage-demo.html>

video element (added many more attributes):

<https://html.spec.whatwg.org/#video>

- audio element:

<https://html.spec.whatwg.org/#audio>